

TUBES I

Membuat Simulasi Antrian Perjalanan Bus

Soal 2.38 – Simulation Modeling and Analysis

Nama/NIM: M. Hazim R. Prajoda / 13523009

Anggota 2: Fajar Kurniawan / 13523027

Deskripsi Masalah dan Interpretasi Model

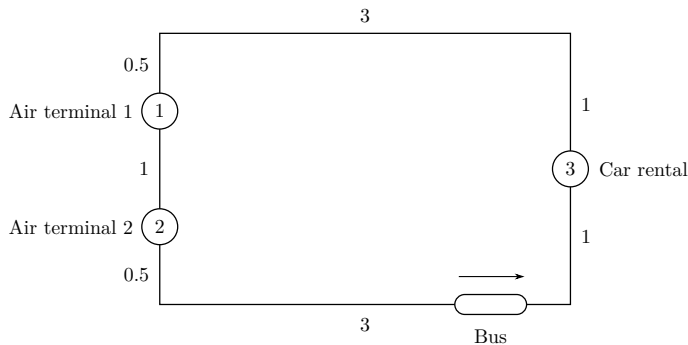


FIGURE 2.72
A car rental system.

Model mensimulasikan layanan bus pada sistem penyewaan mobil yang memiliki tiga lokasi: terminal udara 1, terminal udara 2, dan car rental (lokasi 3). Kedatangan orang di lokasi 1, 2, dan 3 merupakan proses Poisson independen dengan laju berturut-turut 14, 10, dan 24 orang per jam. Setiap lokasi memiliki satu antrian FIFO dengan kapasitas tidak terbatas. Orang yang datang di terminal 1 atau 2 selalu menuju car rental. Orang yang datang di car rental menuju terminal 1 dengan probabilitas 0,583 dan menuju terminal 2 dengan probabilitas 0,417.

Bus berkapasitas 20 orang, bergerak dengan kecepatan 30 mil/jam, dan pada awal simulasi berada di lokasi 3. Bus langsung berangkat berlawanan arah jarum jam dengan urutan lokasi $3 \rightarrow 1 \rightarrow 2 \rightarrow 3$. Berdasarkan jarak pada gambar soal, jarak tempuh antar-lokasi dalam urutan tersebut adalah 4,5 mil, 1 mil, dan 4,5 mil. Dengan demikian, waktu perjalanan bus berturut-turut adalah 0,15 jam, 1/30 jam, dan 0,15 jam.

Saat tiba di suatu lokasi, bus menurunkan semua penumpang tujuan lokasi tersebut secara FIFO, lalu menaikkan orang dari antrian FIFO sampai bus penuh atau antrian habis. Waktu turun per orang berdistribusi uniform 16–24 detik dan waktu naik uniform 15–25 detik. Bus berhenti sekurang-kurangnya 5 menit. Sebelum batas ini, kedatangan baru dapat dilayani saat bus menunggu. Jika batas tercapai saat pelayanan berlangsung, unloading tetap diselesaikan lalu boarding dilanjutkan sampai antrian kosong atau bus penuh; kedatangan selama rangkaian ini dapat ikut dilayani. Sesudah batas 5 menit, bus langsung berangkat ketika rangkaian selesai, tanpa menunggu kedatangan baru.

Simulasi dimulai dari keadaan kosong selama 80 jam tanpa warm-up. Delay diukur sejak kedatangan sampai boarding dimulai; waktu dalam sistem sejak kedatangan sampai penumpang selesai diturunkan, hanya untuk perjalanan selesai sebelum akhir simulasi. Statistik meliputi panjang dan delay antrian, jumlah orang di bus, durasi stop dan loop, serta waktu dalam sistem per lokasi kedatangan. Stop dan loop hanya dicatat jika selesai sebelum akhir simulasi. Keberangkatan awal pada waktu 0 menjadi acuan loop pertama, bukan sampel stop berdurasi nol. Stream SIMLIB 1–3 untuk interarrival lokasi 1–3, stream 4 untuk unloading, stream 5 untuk loading, dan stream 6 untuk tujuan dari lokasi 3.

Berkas model: C dan SIMLIB; parameter demo: 80 jam, laju 14/10/24 orang per jam, kapasitas 20, dan kecepatan 30 mil/jam. Waktu internal dalam jam, hasil waktu ditampilkan dalam menit. CLI: `./tubes1 [input [output]]`; default `tubes1.in/tubes1.out`. Output juga memuat cek kapasitas, konservasi (termasuk orang yang sedang boarding), state, minimum stop/loop, dan horizon; cek internal ini bukan validasi statistik.

Source Code Program (C + SIMLIB)

Source dari `tubes1.c` disajikan berurutan menurut tanggung jawabnya. Nomor baris mengikuti file asli; seluruh kode ditampilkan tanpa perubahan.

A. Setup dan Definisi Model

Library, konstanta, parameter, state bus, pemetaan list, dan deklarasi fungsi.

```

1  /*
2  * TUBES I - Simulasi Antrian Perjalanan Bus
3  * Problem 2.38, Averill M. Law, Simulation Modeling and Analysis.
4  *
5  * Discrete-event simulation implemented in C with SIMLIB.
6  * All simulation time units are hours.
7  */
8
9  #include <ctype.h>
10 #include <errno.h>
11 #include <limits.h>
12 #include <math.h>
13 #include <stdlib.h>
14 #include <sys/stat.h>
15
16 #include "../SIMLIB/simlib.h"
17
18 #define NUM_LOCATIONS          3
19 #define BUS_CAPACITY_DEFAULT   20
20 #define BUS_SPEED_DEFAULT      30.0
21 #define MIN_STOP_TIME          (5.0 / 60.0)
22 #define SECONDS_PER_HOUR       3600.0
23 #define MINUTES_PER_HOUR       60.0
24
25 /* Event types. */
26 #define EVENT_ARRIVAL_1        1
27 #define EVENT_ARRIVAL_2        2
28 #define EVENT_ARRIVAL_3        3
29 #define EVENT_BUS_ARRIVAL      4
30 #define EVENT_UNLOAD_COMPLETE  5
31 #define EVENT_LOAD_COMPLETE    6
32 #define EVENT_STOP_TIMER       7
33 #define EVENT_BUS_DEPARTURE    8
34 #define EVENT_END_SIMULATION   9
35
36 /* SIMLIB lists. Lists 1-3 are location queues. */
37 #define LIST_QUEUE_1           1
38 #define LIST_QUEUE_2           2
39 #define LIST_QUEUE_3           3
40 #define LIST_BUS_TO_1          4
41 #define LIST_BUS_TO_2          5
42 #define LIST_BUS_TO_3          6
43
44 /* SIMLIB statistic variables. */
45 #define STAT_BUS_ONBOARD        1
46 #define STAT_QUEUE_DELAY_BASE   0      /* variables 1-3 */
47 #define STAT_SYSTEM_TIME_BASE   3      /* variables 4-6 */
48 #define STAT_STOP_TIME_BASE     6      /* variables 7-9 */
49 #define STAT_LOOP_TIME          10
50
51 #define PHASE_NONE              0
52 #define PHASE_UNLOADING         1
53 #define PHASE_LOADING           2
54 #define PHASE_WAITING           3
55 #define PHASE_DEPARTURE_SCHEDULED 4
56
57 #define EPS                     1.0e-9
58
59 /* Input and output. */
60 static FILE *infile;
61 static FILE *outfile;
62
63 /* Model parameters. */
64 static double length_simulation;
65 static double arrival_rate[NUM_LOCATIONS + 1];
66 static int bus_capacity;
67 static double bus_speed;
68
69 /* Bus state. */
70 static int bus_active;
71 static int current_stop;
72 static int bus_phase;
73 static int minimum_stop_elapsed;
74 static int onboard;
75 static int seats_reserved_for_loading;
76 static double stop_arrival_time;
77 static double minimum_departure_time;
78 static double last_car_rental_departure;

```

```

79 static long long total_arrivals;
80 static long long total_completed;
81 static long long checked_events;
82 static int state_checks_passed = 1;
83
84 static const char *location_name[NUM_LOCATIONS + 1] = {
85     "", "Air terminal 1", "Air terminal 2", "Car rental"
86 };
87
88 /* Counterclockwise route: 3 -> 1 -> 2 -> 3. */
89 static const int next_stop[NUM_LOCATIONS + 1] = {0, 2, 3, 1};
90 static const double distance_to_next[NUM_LOCATIONS + 1] = {0.0, 1.0, 4.5, 4.5};
91
92 /* Destination probabilities for arrivals at the car rental. */
93 static double destination_distribution[26];
94
95 static int queue_list(int location)
96 {
97     return location;
98 }
99
100 static int bus_list_for_destination(int destination)
101 {
102     return 3 + destination;
103 }
104
105 static void schedule_arrival(int location);
106 static void schedule_bus_event(double time, int event_type, int location);
107 static void begin_unloading(void);
108 static void begin_loading_or_waiting(void);
109 static void start_loading(void);
110 static void maybe_schedule_departure(void);
111 static void handle_person_arrival(int location);
112 static void handle_bus_arrival(void);
113 static void handle_unload_complete(void);
114 static void handle_load_complete(void);
115 static void handle_stop_timer(void);
116 static void handle_bus_departure(void);
117 static void report(void);
118 static int validate_input(void);
119 static int verify_state(void);
120 static int report_verification(void);

```

B. Pembacaan dan Validasi Input

Pembacaan token utuh serta pemeriksaan format dan domain parameter.

```

122 /* Read one complete whitespace-delimited token; reject oversized tokens. */
123 static int read_input_token(char *token, size_t size)
124 {
125     int character;
126     size_t length = 0;
127
128     do {
129         character = fgetc(infile);
130     } while (character != EOF && isspace((unsigned char) character));
131
132     if (character == EOF) {
133         return 0;
134     }
135     do {
136         if (length + 1 >= size) {
137             return 0;
138         }
139         token[length++] = (char) character;
140         character = fgetc(infile);
141     } while (character != EOF && !isspace((unsigned char) character));
142     token[length] = '\0';
143     return !ferror(infile);
144 }
145
146 static int read_input_double(double *value)
147 {
148     char token[128];
149     char *end;
150
151     if (!read_input_token(token, sizeof token)) {
152         return 0;
153     }
154     errno = 0;
155     *value = strtod(token, &end);
156     return errno == 0 && end != token && *end == '\0';
157 }
158
159 static int validate_input(void)
160 {
161     int location;

```

```

162
163     if (!isfinite(length_simulation) || length_simulation <= 0.0) {
164         fprintf(stderr,
165             "Invalid simulation duration: must be finite and greater than zero.\n");
166         return 0;
167     }
168
169     for (location = 1; location <= NUM_LOCATIONS; ++location) {
170         if (!isfinite(arrival_rate[location]) || arrival_rate[location] <= 0.0) {
171             fprintf(stderr,
172                 "Invalid arrival rate at location %d: must be finite and greater than zero.\n",
173                 location);
174             return 0;
175         }
176     }
177
178     if (bus_capacity <= 0) {
179         fprintf(stderr,
180             "Invalid bus capacity: must be a positive integer.\n");
181         return 0;
182     }
183
184     if (!isfinite(bus_speed) || bus_speed <= 0.0) {
185         fprintf(stderr,
186             "Invalid bus speed: must be finite and greater than zero.\n");
187         return 0;
188     }
189
190     return 1;
191 }

```

C. Fungsi Simulasi dan Penanganan Event

Penjadwalan kedatangan, unloading, boarding, waktu berhenti, dan keberangkatan bus.

```

193 static void schedule_arrival(int location)
194 {
195     const int event_type = EVENT_ARRIVAL_1 + location - 1;
196     const double mean_interarrival = 1.0 / arrival_rate[location];
197
198     event_schedule(sim_time + expon(mean_interarrival, location), event_type);
199 }
200
201 static void schedule_bus_event(double time, int event_type, int location)
202 {
203     transfer[3] = (double) location;
204     event_schedule(time, event_type);
205 }
206
207 static void begin_unloading(void)
208 {
209     const int list = bus_list_for_destination(current_stop);
210
211     if (list_size[list] > 0) {
212         bus_phase = PHASE_UNLOADING;
213         transfer[3] = (double) current_stop;
214         event_schedule(sim_time +
215             uniform(16.0 / SECONDS_PER_HOUR,
216                 24.0 / SECONDS_PER_HOUR, 4),
217             EVENT_UNLOAD_COMPLETE);
218     } else {
219         begin_loading_or_waiting();
220     }
221 }
222
223 static void begin_loading_or_waiting(void)
224 {
225     bus_phase = PHASE_WAITING;
226     if (list_size[queue_list(current_stop)] > 0 &&
227         onboard + seats_reserved_for_loading < bus_capacity) {
228         start_loading();
229     } else {
230         maybe_schedule_departure();
231     }
232 }
233
234 static void start_loading(void)
235 {
236     double person_arrival_time;
237     int destination;
238
239     if (!bus_active || bus_phase == PHASE_UNLOADING ||
240         bus_phase == PHASE_LOADING ||
241         list_size[queue_list(current_stop)] == 0 ||
242         onboard + seats_reserved_for_loading >= bus_capacity) {
243         return;
244     }

```

```

245
246 /* Remove the person when boarding starts: this is the queue-delay epoch. */
247 list_remove(FIRST, queue_list(current_stop));
248 person_arrival_time = transfer[1];
249 destination = (int) transfer[3];
250 sampst(sim_time - person_arrival_time,
251        STAT_QUEUE_DELAY_BASE + current_stop);
252
253 ++seats_reserved_for_loading;
254 bus_phase = PHASE_LOADING;
255
256 /* Attributes 3-5 are carried by the load-completion event. */
257 transfer[3] = (double) destination;
258 transfer[4] = person_arrival_time;
259 transfer[5] = (double) current_stop;
260 event_schedule(sim_time +
261               uniform(15.0 / SECONDS_PER_HOUR,
262                      25.0 / SECONDS_PER_HOUR, 5),
263               EVENT_LOAD_COMPLETE);
264 }
265
266 static void maybe_schedule_departure(void)
267 {
268     if (bus_active && bus_phase == PHASE_WAITING &&
269         sim_time + EPS >= minimum_departure_time) {
270         bus_phase = PHASE_DEPARTURE_SCHEDULED;
271         schedule_bus_event(sim_time, EVENT_BUS_DEPARTURE, current_stop);
272     }
273 }
274
275 static void handle_person_arrival(int location)
276 {
277     int destination;
278
279     /* Keep one independent exponential arrival process at each location. */
280     schedule_arrival(location);
281
282     transfer[1] = sim_time; /* Arrival time in the system. */
283     transfer[2] = (double) location;
284     if (location == 3) {
285         destination = random_integer(destination_distribution, 6);
286     } else {
287         destination = 3;
288     }
289     transfer[3] = (double) destination;
290     list_file(LAST, queue_list(location));
291     ++total_arrivals;
292
293     /* A person arriving while the bus is waiting may start a new loading
294        operation, provided the five-minute minimum stop has not expired. */
295     if (bus_active && current_stop == location &&
296         bus_phase == PHASE_WAITING && !minimum_stop_elapsed &&
297         onboard + seats_reserved_for_loading < bus_capacity) {
298         start_loading();
299     }
300 }
301
302 static void handle_bus_arrival(void)
303 {
304     current_stop = (int) transfer[3];
305     bus_active = 1;
306     bus_phase = PHASE_UNLOADING;
307     minimum_stop_elapsed = 0;
308     stop_arrival_time = sim_time;
309     minimum_departure_time = sim_time + MIN_STOP_TIME;
310
311     /* The timer lets arrivals during the mandatory five-minute dwell start
312        loading if the bus is otherwise idle. */
313     schedule_bus_event(minimum_departure_time, EVENT_STOP_TIMER, current_stop);
314     begin_unloading();
315 }
316
317 static void handle_unload_complete(void)
318 {
319     const int list = bus_list_for_destination(current_stop);
320     int origin;
321
322     list_remove(FIRST, list);
323     origin = (int) transfer[2];
324     sampst(sim_time - transfer[1], STAT_SYSTEM_TIME_BASE + origin);
325     --onboard;
326     ++total_completed;
327     timest((double) onboard, STAT_BUS_ONBOARD);
328
329     if (list_size[list] > 0) {
330         transfer[3] = (double) current_stop;
331         event_schedule(sim_time +
332                       uniform(16.0 / SECONDS_PER_HOUR,
333                              24.0 / SECONDS_PER_HOUR, 4),
334                       EVENT_UNLOAD_COMPLETE);

```

```

335 } else {
336     begin_loading_or_waiting();
337 }
338 }
339
340 static void handle_load_complete(void)
341 {
342     const int destination = (int) transfer[3];
343     const int origin = (int) transfer[5];
344
345     transfer[1] = transfer[4];          /* Restore passenger attributes. */
346     transfer[2] = (double) origin;
347     transfer[3] = (double) destination;
348     list_file(LAST, bus_list_for_destination(destination));
349
350     --seats_reserved_for_loading;
351     ++onboard;
352     timest((double) onboard, STAT_BUS_ONBOARD);
353
354     bus_phase = PHASE_WAITING;
355     if (list_size[queue_list(current_stop)] > 0 &&
356         onboard + seats_reserved_for_loading < bus_capacity) {
357         start_loading();
358     } else {
359         maybe_schedule_departure();
360     }
361 }
362
363 static void handle_stop_timer(void)
364 {
365     minimum_stop_elapsed = 1;
366
367     if (bus_phase == PHASE_UNLOADING || bus_phase == PHASE_LOADING) {
368         /* The active operation is allowed to finish before departure. */
369         return;
370     }
371
372     if (bus_phase == PHASE_WAITING) {
373         if (list_size[queue_list(current_stop)] > 0 &&
374             onboard + seats_reserved_for_loading < bus_capacity) {
375             start_loading();
376         } else {
377             maybe_schedule_departure();
378         }
379     }
380 }
381
382 static void handle_bus_departure(void)
383 {
384     const int departed_from = current_stop;
385     const int next = next_stop[departed_from];
386     const double stop_duration = sim_time - stop_arrival_time;
387
388     sampst(stop_duration, STAT_STOP_TIME_BASE + departed_from);
389     if (departed_from == 3) {
390         sampst(sim_time - last_car_rental_departure, STAT_LOOP_TIME);
391         last_car_rental_departure = sim_time;
392     }
393
394     bus_active = 0;
395     bus_phase = PHASE_NONE;
396     current_stop = 0;
397
398     schedule_bus_event(sim_time +
399                       distance_to_next[departed_from] / bus_speed,
400                       EVENT_BUS_ARRIVAL, next);
401 }

```

D. Statistik, Output, dan Verifikasi

Penyajian statistik dalam menit serta pemeriksaan konsistensi internal model.

```

403 static void print_sampst_row(int variable, const char *label)
404 {
405     const double average = sampst(0.0, -variable);
406     const double observations = transfer[2];
407     const double maximum = observations == 0.0 ? 0.0 : transfer[3];
408     const double minimum = observations == 0.0 ? 0.0 : transfer[4];
409
410     fprintf(outfile, "%-18s %10.4f %12.0f %10.4f %10.4f\n",
411             label, average * MINUTES_PER_HOUR, observations,
412             maximum * MINUTES_PER_HOUR, minimum * MINUTES_PER_HOUR);
413 }
414
415 static void print_queue_row(int location)
416 {
417     double average_number;

```

```

418 double maximum_number;
419 double average_delay;
420 double delay_observations;
421 double maximum_delay;
422 double minimum_delay;
423
424 filest(location);
425 average_number = transfer[1];
426 maximum_number = transfer[2] < 0.0 ? 0.0 : transfer[2];
427
428 average_delay = sampst(0.0, -location);
429 delay_observations = transfer[2];
430 maximum_delay = delay_observations == 0.0 ? 0.0 : transfer[3];
431 minimum_delay = delay_observations == 0.0 ? 0.0 : transfer[4];
432
433 fprintf(outfile, "%-18s %10.4f %10.0f %12.4f %12.0f %10.4f %10.4f\n",
434         location_name[location], average_number, maximum_number,
435         average_delay * MINUTES_PER_HOUR, delay_observations,
436         maximum_delay * MINUTES_PER_HOUR, minimum_delay * MINUTES_PER_HOUR);
437 }
438
439 static void report(void)
440 {
441     double average_onboard;
442     double maximum_onboard;
443     double average_loop;
444     double loop_observations;
445     double maximum_loop;
446     double minimum_loop;
447
448     fprintf(outfile, "Car-rental bus simulation using SIMLIB\n");
449     fprintf(outfile, "Problem 2.38 -- %g-hour run\n\n", length_simulation);
450     fprintf(outfile, "Time unit: minute (internal simulation: hour). Distance unit: mile.\n");
451     fprintf(outfile, "Locations and arrival rates:\n");
452     fprintf(outfile, " 1 %-18s %8.3f people/hour\n", location_name[1], arrival_rate[1]);
453     fprintf(outfile, " 2 %-18s %8.3f people/hour\n", location_name[2], arrival_rate[2]);
454     fprintf(outfile, " 3 %-18s %8.3f people/hour\n", location_name[3], arrival_rate[3]);
455     fprintf(outfile, "Bus capacity: %d people; speed: %.3f miles/hour\n", bus_capacity, bus_speed);
456     fprintf(outfile, "Counterclockwise route: 3 -> 1 -> 2 -> 3\n");
457     fprintf(outfile, "Travel distances: 3->1 = %.1f, 1->2 = %.1f, 2->3 = %.1f miles\n\n",
458             distance_to_next[3], distance_to_next[1], distance_to_next[2]);
459
460     fprintf(outfile, "(a)-(b) Queue length and queue-delay statistics\n");
461     fprintf(outfile, "Location      Avg #      Max #      Avg delay (min) N delay  Max delay  Min delay\n");
462     fprintf(outfile, "-----\n");
463     print_queue_row(1);
464     print_queue_row(2);
465     print_queue_row(3);
466     fprintf(outfile, "\n");
467
468     fprintf(outfile, "(c) Number of people on the bus\n");
469     average_onboard = timest(0.0, -STAT_BUS_ONBOARD);
470     maximum_onboard = transfer[2];
471     fprintf(outfile, "Time-average number on bus: %.4f people\n", average_onboard);
472     fprintf(outfile, "Maximum number on bus:      %.0f people\n\n", maximum_onboard);
473
474     fprintf(outfile, "(d) Bus stop duration statistics (minutes)\n");
475     fprintf(outfile, "Location      Average      N      Maximum      Minimum\n");
476     fprintf(outfile, "-----\n");
477     print_sampst_row(STAT_STOP_TIME_BASE + 1, location_name[1]);
478     print_sampst_row(STAT_STOP_TIME_BASE + 2, location_name[2]);
479     print_sampst_row(STAT_STOP_TIME_BASE + 3, location_name[3]);
480     fprintf(outfile, "\n");
481
482     fprintf(outfile, "(e) Bus loop duration (departure from location 3 to next departure)\n");
483     average_loop = sampst(0.0, -STAT_LOOP_TIME);
484     loop_observations = transfer[2];
485     maximum_loop = loop_observations == 0.0 ? 0.0 : transfer[3];
486     minimum_loop = loop_observations == 0.0 ? 0.0 : transfer[4];
487     fprintf(outfile, "Average: %.4f min; N: %.0f; Maximum: %.4f min; Minimum: %.4f min\n\n",
488             average_loop * MINUTES_PER_HOUR, loop_observations,
489             maximum_loop * MINUTES_PER_HOUR, minimum_loop * MINUTES_PER_HOUR);
490
491     fprintf(outfile, "(f) Time in system by arrival location (completed trips only)\n");
492     fprintf(outfile, "Arrival location  Average (min)  N      Maximum      Minimum\n");
493     fprintf(outfile, "-----\n");
494     print_sampst_row(STAT_SYSTEM_TIME_BASE + 1, location_name[1]);
495     print_sampst_row(STAT_SYSTEM_TIME_BASE + 2, location_name[2]);
496     print_sampst_row(STAT_SYSTEM_TIME_BASE + 3, location_name[3]);
497     fprintf(outfile, "\n");
498
499     fprintf(outfile, "Simulation ended at %.4f min (%.4f h).\n",
500             sim_time * MINUTES_PER_HOUR, sim_time);
501     fprintf(outfile, "Queue delay is measured when boarding starts; time in system is\n");
502     fprintf(outfile, "reported when a passenger is unloaded at the destination.\n");
503 }
504
505 /* Check after each handled event, including passengers currently boarding. */
506 static int verify_state(void)
507 {

```

```

508     long long queued = 0, bus_records = 0;
509     int location;
510
511     for (location = 1; location <= NUM_LOCATIONS; ++location) {
512         queued += list_size[queue_list(location)];
513         bus_records += list_size[bus_list_for_destination(location)];
514     }
515     return onboard == bus_records && onboard >= 0 &&
516         seats_reserved_for_loading >= 0 && seats_reserved_for_loading <= 1 &&
517         (long long) onboard + seats_reserved_for_loading <= bus_capacity &&
518         (bus_phase == PHASE_LOADING) == (seats_reserved_for_loading == 1) &&
519         total_arrivals == total_completed + queued + bus_records +
520             seats_reserved_for_loading;
521 }
522
523 static int report_verification(void)
524 {
525     long long queued = 0;
526     double expected_loop = 3.0 * MIN_STOP_TIME;
527     double minimum_loop, loop_samples, stop_samples = 0.0;
528     double maximum_onboard;
529     int location, capacity_ok, conservation_ok, loop_ok = 1, stop_ok = 1;
530     int horizon_ok = sim_time == length_simulation;
531     int passed;
532
533     for (location = 1; location <= NUM_LOCATIONS; ++location) {
534         queued += list_size[queue_list(location)];
535         expected_loop += distance_to_next[location] / bus_speed;
536         sampst(0.0, -(STAT_STOP_TIME_BASE + location));
537         stop_samples += transfer[2];
538         if (transfer[2] > 0.0 && transfer[4] + EPS < MIN_STOP_TIME) {
539             stop_ok = 0;
540         }
541     }
542     timest(0.0, -STAT_BUS_ONBOARD);
543     maximum_onboard = transfer[2];
544     capacity_ok = maximum_onboard <= bus_capacity;
545     conservation_ok = total_arrivals == total_completed + queued + onboard +
546         seats_reserved_for_loading;
547     sampst(0.0, -STAT_LOOP_TIME);
548     loop_samples = transfer[2];
549     minimum_loop = loop_samples > 0.0 ? transfer[4] : 0.0;
550     if (loop_samples > 0.0) {
551         loop_ok = minimum_loop + EPS >= expected_loop;
552     }
553     fprintf(outfile, "\nInternal verification\n");
554     fprintf(outfile, "[%s] Capacity: maximum %.0f; limit %d people\n",
555         capacity_ok ? "PASS" : "FAIL", maximum_onboard, bus_capacity);
556     fprintf(outfile, "[%s] Conservation: arrivals=%lld, completed=%lld,\n",
557         conservation_ok ? "PASS" : "FAIL", total_arrivals, total_completed);
558     fprintf(outfile, "        queued=%lld, onboard=%d, boarding=%d\n",
559         queued, onboard, seats_reserved_for_loading);
560     fprintf(outfile, "[%s] State invariants: %lld handled events checked\n",
561         state_checks_passed ? "PASS" : "FAIL", checked_events);
562     if (stop_samples > 0.0) {
563         fprintf(outfile, "[%s] Minimum stop: completed stops >= %.4f min\n",
564             stop_ok ? "PASS" : "FAIL", MIN_STOP_TIME * MINUTES_PER_HOUR);
565     } else {
566         fprintf(outfile, "[SKIP] Minimum stop: no completed stop\n");
567     }
568     if (loop_samples > 0.0) {
569         fprintf(outfile, "[%s] Minimum loop: %.4f min; bound %.4f min\n",
570             loop_ok ? "PASS" : "FAIL", minimum_loop * MINUTES_PER_HOUR,
571             expected_loop * MINUTES_PER_HOUR);
572     } else {
573         fprintf(outfile, "[SKIP] Minimum loop: no completed loop\n");
574     }
575     fprintf(outfile, "[%s] Simulation horizon: %.4f min\n",
576         horizon_ok ? "PASS" : "FAIL", sim_time * MINUTES_PER_HOUR);
577     passed = capacity_ok && conservation_ok && state_checks_passed &&
578         stop_ok && loop_ok && horizon_ok;
579     fprintf(outfile, "%s (not statistical validation).\n",
580         passed ? "All internal checks passed" : "Internal checks failed");
581     return passed;
582 }

```

E. Program Utama

CLI input/output, pembacaan parameter, inisialisasi, event loop, dan pelaporan akhir.

```

584 int main(int argc, char *argv[])
585 {
586     int event_type;
587     int verification_passed;
588     char trailing_character;
589     char capacity_token[128];
590     char *capacity_end;
591     long long capacity_input;
592     const char *input_path = argc > 1 ? argv[1] : "tubes1.in";
593     const char *output_path = argc > 2 ? argv[2] : "tubes1.out";
594     struct stat input_status, output_status;
595
596     if (argc > 3) {
597         fprintf(stderr, "Usage: %s [input_file [output_file]]\n", argv[0]);
598         return 1;
599     }
600     if (stat(input_path, &input_status) == 0 &&
601         stat(output_path, &output_status) == 0 &&
602         input_status.st_dev == output_status.st_dev &&
603         input_status.st_ino == output_status.st_ino) {
604         fprintf(stderr, "Input and output must refer to different files.\n");
605         return 1;
606     }
607     infile = fopen(input_path, "r");
608     if (infile == NULL) {
609         fprintf(stderr, "Cannot open input: %s\n", input_path);
610         return 1;
611     }
612
613     if (!read_input_double(&length_simulation) ||
614         !read_input_double(&arrival_rate[1]) ||
615         !read_input_double(&arrival_rate[2]) ||
616         !read_input_double(&arrival_rate[3]) ||
617         !read_input_token(capacity_token, sizeof capacity_token) ||
618         !read_input_double(&bus_speed) ||
619         fscanf(infile, " %c", &trailing_character) == 1 || ferror(infile)) {
620         fprintf(stderr, "Invalid input format: %s\n", input_path);
621         fclose(infile);
622         return 1;
623     }
624     fclose(infile);
625
626     errno = 0;
627     capacity_input = strtoll(capacity_token, &capacity_end, 10);
628     if (errno != 0 || capacity_end == capacity_token || *capacity_end != '\0' ||
629         capacity_input < INT_MIN || capacity_input > INT_MAX) {
630         fprintf(stderr, "Invalid bus capacity: must be an integer within int range.\n");
631         return 1;
632     }
633     bus_capacity = (int) capacity_input;
634     if (!validate_input()) {
635         return 1;
636     }
637
638     outfile = fopen(output_path, "w");
639     if (outfile == NULL) {
640         fprintf(stderr, "Cannot open output: %s\n", output_path);
641         return 1;
642     }
643
644     destination_distribution[1] = 0.583;
645     destination_distribution[2] = 1.000;
646
647     bus_active = 0;
648     current_stop = 0;
649     bus_phase = PHASE_NONE;
650     minimum_stop_elapsed = 0;
651     onboard = 0;
652     seats_reserved_for_loading = 0;
653     last_car_rental_departure = 0.0;
654
655     /* maxatr must cover the passenger attributes carried by events. */
656     maxatr = 5;
657     init_simlib();
658     timest(0.0, STAT_BUS_ONBOARD);
659
660     /* Three independent arrival processes. */
661     schedule_arrival(1);
662     schedule_arrival(2);
663     schedule_arrival(3);
664
665     /* The bus begins at location 3 and leaves immediately counterclockwise. */
666     schedule_bus_event(distance_to_next[3] / bus_speed,
667                       EVENT_BUS_ARRIVAL, 1);
668     event_schedule(length_simulation, EVENT_END_SIMULATION);
669

```

```

670 while (1) {
671     timing();
672     event_type = next_event_type;
673
674     if (event_type == EVENT_END_SIMULATION) {
675         break;
676     }
677
678     switch (event_type) {
679     case EVENT_ARRIVAL_1:
680         handle_person_arrival(1);
681         break;
682     case EVENT_ARRIVAL_2:
683         handle_person_arrival(2);
684         break;
685     case EVENT_ARRIVAL_3:
686         handle_person_arrival(3);
687         break;
688     case EVENT_BUS_ARRIVAL:
689         handle_bus_arrival();
690         break;
691     case EVENT_UNLOAD_COMPLETE:
692         handle_unload_complete();
693         break;
694     case EVENT_LOAD_COMPLETE:
695         handle_load_complete();
696         break;
697     case EVENT_STOP_TIMER:
698         handle_stop_timer();
699         break;
700     case EVENT_BUS_DEPARTURE:
701         handle_bus_departure();
702         break;
703     default:
704         fprintf(stderr, "Unknown event type %d at time %.6f\n",
705             event_type, sim_time);
706         fclose(outfile);
707         return 1;
708     }
709     ++checked_events;
710     state_checks_passed &= verify_state();
711 }
712
713 report();
714 state_checks_passed &= verify_state();
715 verification_passed = report_verification();
716 if (fclose(outfile) != 0) {
717     fprintf(stderr, "Cannot finish writing output: %s\n", output_path);
718     return 1;
719 }
720 return verification_passed ? 0 : 1;
721 }

```

Output Program

```

1 Car-rental bus simulation using SIMLIB
2 Problem 2.38 -- 80-hour run
3
4 Time unit: minute (internal simulation: hour). Distance unit: mile.
5 Locations and arrival rates:
6   1 Air terminal 1      14.000 people/hour
7   2 Air terminal 2      10.000 people/hour
8   3 Car rental         24.000 people/hour
9 Bus capacity: 20 people; speed: 30.000 miles/hour
10 Counterclockwise route: 3 -> 1 -> 2 -> 3
11 Travel distances: 3->1 = 4.5, 1->2 = 1.0, 2->3 = 4.5 miles
12
13 (a)-(b) Queue length and queue-delay statistics
14 Location          Avg #      Max #      Avg delay (min) N delay  Max delay  Min delay
15 -----
16 Air terminal 1      7.3677      27      31.4140      1120    91.9342    0.0000
17 Air terminal 2      4.8875      17      29.8196      784    130.8598    0.0000
18 Car rental         9.3210      29      24.1302     1854    67.4846    0.0125
19
20 (c) Number of people on the bus
21 Time-average number on bus: 13.8530 people
22 Maximum number on bus:      20 people
23
24 (d) Bus stop duration statistics (minutes)
25 Location          Average      N      Maximum      Minimum
26 -----
27 Air terminal 1      6.6982      110    10.1706      5.0000
28 Air terminal 2      5.4732      110     9.2866      5.0000
29 Car rental        11.4018      110    13.7431     5.5718
30
31 (e) Bus loop duration (departure from location 3 to next departure)
32 Average: 43.5733 min; N: 110; Maximum: 48.0692 min; Minimum: 35.7146 min
33
34 (f) Time in system by arrival location (completed trips only)
35 Arrival location  Average (min)      N      Maximum      Minimum
36 -----
37 Air terminal 1      51.9273      1120    112.2936     17.7385
38 Air terminal 2      45.5444      784     147.0041     13.9285
39 Car rental         41.7239     1838     88.2891     11.5316
40
41 Simulation ended at 4800.0000 min (80.0000 h).
42 Queue delay is measured when boarding starts; time in system is
43 reported when a passenger is unloaded at the destination.
44
45 Internal verification
46 [PASS] Capacity: maximum 20; limit 20 people
47 [PASS] Conservation: arrivals=3778, completed=3742,
48      queued=20, onboard=16, boarding=0
49 [PASS] State invariants: 12268 handled events checked
50 [PASS] Minimum stop: completed stops >= 5.0000 min
51 [PASS] Minimum loop: 35.7146 min; bound 35.0000 min
52 [PASS] Simulation horizon: 4800.0000 min
53 All internal checks passed (not statistical validation).

```